Multimedia Mini Project Assignment

Simplified MPEG-4 Video Encoder Pipeline

Multimedia Systems

April 13, 2026

Introduction

Video compression is at the heart of modern multimedia systems. Every video you stream, record, or share relies on a **codec**: a system that encodes raw visual data into a compact bitstream and decodes it back into images.

Project Objectives: Implementing a **simplified** but **complete** MPEG-4-like encoder pipeline in Python, from raw image frames to a compressed binary file that can be decoded back into images.

Project Overview

Input: A folder of sequential image frames (`.png` or `.jpg`) representing a short video clip.
Output: A compressed binary file (`.bin`) encoding the full video, which can be decoded back into the original frames.

The pipeline consists of five stages, each corresponding to a key component of a real video codec:

1. **Pre-processing** — Color space conversion and chroma subsampling
2. **Intra-frame coding (I-frames)** — DCT-based spatial compression
3. **Inter-frame coding (P-frames)** — Motion estimation and residual coding
4. **Entropy coding** — Lossless compression of the bitstream
5. **Evaluation & Visualisation** — Quality metrics and pipeline visualisation

Requirements

Part 1 — Pre-processing (10%)

Convert each input BGR frame into the **YCbCr color space**

Part 2 — Intra-frame Coding / I-frames (25%)

Implement DCT-based compression on individual frames (no reference to other frames)
The **decoder** must invert this process: dequantise (multiply by the quantisation matrix) then apply the inverse DCT (IDCT).

Part 3 — Inter-frame Coding / P-frames (25%)

Exploit temporal redundancy between consecutive frames:

- Define a **Group of Pictures (GOP)** size G. Every G-th frame is coded as an I-frame; the remaining frames are P-frames.
- For each P-frame, divide the Y channel into 16×16 macroblocks.
- For each macroblock, perform **block matching** against the previous reconstructed frame within a search window of $\pm S$ pixels.
- Record the resulting **motion vector** for each macroblock.
- Compute the **residual** (difference between the current block and its prediction) and apply DCT + quantisation to the residual only.

The decoder reconstructs each P-frame by combining the motion-compensated prediction from the reference frame with the decoded residual.

Part 4 — Entropy Coding (15%)

Apply lossless compression to reduce the size of the stored coefficients and motion vectors.

- Apply **lossless compression** to the serialised data.
- Write the result to a `.bin` output file.
- Implement a corresponding **decoder**.

Part 5 — Evaluation & Visualisation (25%)

5a — Quality Metrics

Compute the following for each reconstructed frame:

- **Compression ratio**: ratio $= \frac{\text{original size (bytes)}}{\text{compressed size (bytes)}}$
- Frame-type breakdown (number of I-frames vs P-frames).

5b — Pipeline Visualisation

Using `matplotlib`, produce a single figure that visualises **every stage** of the pipeline:

1. **Original frames** — display the input frame sequence.
2. **Color space** — show the Y, Cb, Cr channels of one frame separately.
3. **DCT & Quantisation** — show one 8×8 block going through: raw pixels $\rightarrow$ DCT coefficients $\rightarrow$ quantised coefficients $\rightarrow$ reconstructed block.

4. **Motion vectors** — overlay motion vector arrows on a P-frame.
5. **Residuals & Reconstruction** — show residual maps alongside reconstructed frames.

Deliverables

Each **group of 2 students** must host their project on a public **GitHub repository**. The repository must contain all required source files, the compressed output, and the **written report.**

Written Report (included in grading)

Submit a `report.pdf` as part of your repository. The report must include the following sections:

a. **Pipeline Description**

b. **Design Choices & Justification**

c. **Experimental Analysis**
   - Plot compression ratio vs `Quantization Factor`.
   - Show how **GOP** size affects the compression.

Good luck!
Please contact us with any questions or clarifications.